

## -- HP紹介内容 --

---

## 研修名

---

C#テストコード基礎

- MSTestで学ぶユニットテストと結合テスト -

## キャッチコピー

---

テストを書く習慣が、コードの品質を変える。C#で学ぶ、実務テストコードの基礎。

## 研修概要

---

### ■ 本研修について

本研修では、C#によるテストコードの基礎を体系的に学びます。MSTestを用いたユニットテストの作成から、テスト用属性・例外処理のテスト・カバレッジ計測・Moqを用いた結合テストまで、実務に必要なテストコードのスキルを習得します。

### ■ 本研修の3つの特徴

 テストケース設計の思考プロセスまで学ぶ 境界値・同値分割・代表値を意識したテストケース表（観点・入力・期待結果・理由）の作成方法を習得します。「なぜそのテストケースを選ぶか」まで理解できます。  
 MSTestの実務属性を完全習得 DataRow・DataTestMethod・TestInitialize・TestCleanup・TestCategoryなど、実務でよく使うMSTestの属性を網羅的に習得します。  
 Moqによる結合テストとカバレッジ計測まで一貫して学ぶ インターフェ이스のモック生成・Verify検証によるMoqの結合テストと、カバレッジ計測レポートの取得・可視化まで習得します。

### ■ 到達目標（受講後に身につくスキル）

・境界値・同値分割・代表値を意識したテストケースを設計・作成できる ・MSTestを用いてユニットテスト・例外処理テストを実装できる ・Moqを用いたインターフェースのモックと結合テストを実装できる

## こんな企業にお勧め

---

- C#エンジニアのテストコードスキルにばらつきがあり、チーム全体に標準化したい
- テストを書く習慣がなく、品質管理をテスト自動化で改善したい
- 新卒・若手C#エンジニアにテストコードの基礎を体系的に習得させたい
- 「コードは書けるがテストが書けない」エンジニアのスキルアップを図りたい
- テストケース設計の考え方をエンジニア全員に身につけさせたい

## 日数

1日

# カリキュラム詳細（全1日間）

| 日程          | 章・学習テーマ            | 学習内容・習得スキル                                                                                                       |
|-------------|--------------------|------------------------------------------------------------------------------------------------------------------|
| 1<br>日<br>目 | 第1章：MSTestの概要      | MSTestの概要・テストの種類（ユニット・統合・E2E）・MSTestプロジェクトの構成を理解する。                                                              |
|             | 第2章：ユニットテスト（単体テスト） | テストの目的・手順・テストターゲットの実装・テストケース設計（境界値・同値分割・代表値）・テストドライバの作成・Assertメソッド・テストの実行と結果確認を習得する。                             |
|             | 第3章：実務でよく使うテスト用属性  | DataRow・DataTestMethod・TestInitialize・TestCleanup・ClassInitialize・Ignore・TestCategoryなどの実務属性を習得し、効率的なテスト実装を実践する。 |
|             | 第4章：例外処理のテスト       | ExpectedException属性・Assert.ThrowsException・例外メッセージの検証を習得し、異常系テストを実装する。                                           |
|             | 第5章：カバレッジ          | カバレッジの概念・テスト結果の可視化・カバレッジ計測レポートの取得と読み方を習得する。                                                                      |
|             | 第6章：結合テスト          | モックの概念・Moqによるインターフェイスのモック生成・依存性注入・Verify検証を習得し、結合テストを実装する。                                                       |

※ 各章に演習課題・模範解答・実行結果スクリーンショットが付属しています。

## 前提知識・スキル-> 受講にあたっての前提知識

・C#言語の基礎知識（変数・条件分岐・ループ・クラス・メソッド）を理解していること  
・オブジェクト指向プログラミングの基礎知識（クラス・インターフェイス・継承）を理解していること  
・Visual Studio Codeの基本操作ができること

## 必要な受講環境

| 区分         | 環境 / ツール                           | 役割・用途            |
|------------|------------------------------------|------------------|
| PC         | Windows 11 / macOS                 | 受講用PC（OS問わず）     |
| IDE        | Visual Studio Code                 | コード編集・デバッグ・テスト実行 |
| SDK        | .NET 8                             | C#アプリケーション開発・実行  |
| テストフレームワーク | MSTest（Microsoft.Testing.Platform） | ユニットテスト・結合テストの実行 |

| 区分         | 環境 / ツール          | 役割・用途                |
|------------|-------------------|----------------------|
| モックライブラリ   | Moq               | インターフェイスのモック生成・結合テスト |
| VSCode拡張機能 | C# Dev Kit        | C#コード補完・デバッグ・テスト実行支援 |
| VSCode拡張機能 | .NET Install Tool | .NET SDKのインストール支援    |